End-to-End Data Engineering Pipeline in Snowflake using Python (Snowpark)

This project demonstrates a complete ELT (Extract, Load, Transform) data pipeline using Snowflake Snowpark for Python.

Architecture Overview

1. Setup: Initialize the Snowpark session and create the necessary database, schemas, and warehouses.
2. Ingestion: Simulate loading raw semi-structured JSON/CSV data from an internal stage into a raw table.
3. Transformation: Use Snowpark DataFrames to clean, parse, and transform the raw data.
4. Target Loading: Write the transformed data into a structured dimensional model (dim_customers and fact_orders).
5. Data Quality: Run validation checks directly inside Snowflake.

[1]

```
# Install Snowflake and Snowpark libraries and dependencies
!pip install snowflake
!pip install snowflake-snowpark-python
```

```
Requirement already satisfied: snowflake in /opt/python/cpython-3.12.12-linux-x86_64-gnu/lib/python3.12/site-packages (1.13.1)
Requirement already satisfied: snowflake-core==1.13.1 in /opt/python/cpython-3.12.12-linux-x86_64-gnu/lib/python3.12/site-packages
Requirement already satisfied: snowflake-legacy in /opt/python/cpython-3.12.12-linux-x86_64-gnu/lib/python3.12/site-packages (from
...
Requirement already satisfied: six>=1.5 in /opt/python/cpython-3.12.12-linux-x86_64-gnu/lib/python3.12/site-packages (from python-
Requirement already satisfied: cffi>=2.0.0 in /opt/python/cpython-3.12.12-linux-x86_64-gnu/lib/python3.12/site-packages (from cryp
Requirement already satisfied: pycparser in /opt/python/cpython-3.12.12-linux-x86_64-gnu/lib/python3.12/site-packages (from cffi>=
(pip log abbreviated – all requirements already satisfied)
```

⚠️ **Do not forget to "RESTART RUNTIME" or "RESTART KERNEL"**

---

[1]

```
# Environment Setup & Session Initialization
# (In Snowflake Notebooks, 'get_active_session()' is automatically available.)

from snowflake.snowpark.functions import col, lit, current_timestamp, sum as spark_sum, when, to_date, upper, trim
from snowflake.snowpark.types import StructType, StructField, StringType, IntegerType, FloatType
from snowflake.snowpark.context import get_active_session

session = get_active_session()
```

Step 1: Database and Schema Setup

We will set up a dedicated environment for our data engineering workflow, including raw and analytics schemas.

[2]

```
session.sql("CREATE DATABASE IF NOT EXISTS DE_SAMPLE_DB").collect()
session.sql("CREATE SCHEMA IF NOT EXISTS DE_SAMPLE_DB.RAW").collect()
session.sql("CREATE SCHEMA IF NOT EXISTS DE_SAMPLE_DB.ANALYTICS").collect()

session.sql("USE DATABASE DE_SAMPLE_DB").collect()
print("Database and schemas configured successfully.")
```

```
Database and schemas configured successfully.
```

Step 2: Data Ingestion (Raw Layer)

We'll create a dummy raw table and ingest sample order data simulating an upstream application export.

[3]

```
# Create a sample raw staging table
session.sql("""
    CREATE OR REPLACE TABLE DE_SAMPLE_DB.RAW.raw_orders (
        order_id INT,
        customer_name STRING,
        order_date STRING,
        amount FLOAT,
        status STRING
    )
""").collect()
```

```
[Row(status='Table RAW_ORDERS successfully created.')]
```

[4]

```
# Ensure an active warehouse for DML
session.sql("USE WAREHOUSE SNOWFLAKE_LEARNING_WH").collect()

# Insert mock raw data
session.sql("""
    INSERT INTO DE_SAMPLE_DB.RAW.raw_orders VALUES
    (1, 'Alice Smith', '2026-03-01', 150.50, 'COMPLETED'),
    (2, 'Bob Jones', '2026-03-01', 99.99, 'PENDING'),
    (3, 'Charlie Brown', '2026-03-02', 250.00, 'COMPLETED'),
    (4, 'Alice Smith', '2026-03-03', 45.00, 'CANCELLED'),
    (5, 'Diana Prince', '2026-03-03', 300.00, 'COMPLETED')
"""
).collect()
```

```
[Row(number of rows inserted=5)]
```

[5]

```
raw_df = session.table("DE_SAMPLE_DB.RAW.raw_orders")
raw_df.show()
```

```
-------------------------------------------------------------------------
|"ORDER_ID"  |"CUSTOMER_NAME"  |"ORDER_DATE"  |"AMOUNT"  |"STATUS"   |
-------------------------------------------------------------------------
|1           |Alice Smith      |2026-03-01    |150.5     |COMPLETED  |
|2           |Bob Jones        |2026-03-01    |99.99     |PENDING    |
|3           |Charlie Brown    |2026-03-02    |250.0     |COMPLETED  |
|4           |Alice Smith      |2026-03-03    |45.0      |CANCELLED  |
|5           |Diana Prince     |2026-03-03    |300.0     |COMPLETED  |
-------------------------------------------------------------------------
```

Step 3: Transformation Layer using Snowpark DataFrames

Using Snowpark DataFrames, we parse dates, normalize customer names, filter out cancelled orders, and prepare our dimensional and fact tables.

[7]

```
# Read from raw table
df_raw = session.table("DE_SAMPLE_DB.RAW.raw_orders")
```

[8]

```
# Apply transformations
df_transformed = (
    df_raw
    .filter(col("status") != "CANCELLED")
    .with_column("CUSTOMER_NAME_CLEAN", upper(trim(col("customer_name"))))
    .with_column("ORDER_DATE_Parsed", to_date(col("order_date")))
    .drop("customer_name", "order_date")
)
```

[9]

```
df_transformed.show()
```

```
-------------------------------------------------------------------------------
|"ORDER_ID"  |"AMOUNT"  |"STATUS"   |"CUSTOMER_NAME_CLEAN"  |"ORDER_DATE_PARSED"  |
-------------------------------------------------------------------------------
|1           |150.5     |COMPLETED  |ALICE SMITH            |2026-03-01           |
|2           |99.99     |PENDING    |BOB JONES              |2026-03-01           |
|3           |250.0     |COMPLETED  |CHARLIE BROWN          |2026-03-02           |
|5           |300.0     |COMPLETED  |DIANA PRINCE           |2026-03-03           |
-------------------------------------------------------------------------------
```

Step 4: Loading Target Tables (Dimensional Model)

We split the transformed stream into a customer dimension table (dim_customers) and an orders fact table (fact_orders), using write.save_as_table for idempotent or append loads.

[10]

```
# Create Dim Customers (deduplicated)
df_customers = (
    df_transformed
    .select(col("CUSTOMER_NAME_CLEAN").alias("CUSTOMER_NAME"))
    .distinct()
)

df_customers.write.mode("overwrite").save_as_table("DE_SAMPLE_DB.ANALYTICS.dim_customers")
```

[11]

```
# Create Fact Orders
df_orders = (
    df_transformed
    .select(
        col("order_id"),
        col("CUSTOMER_NAME_CLEAN").alias("customer_name"),
        col("ORDER_DATE_Parsed").alias("order_date"),
        col("amount"),
```

```
        col("status")
    )
)

df_orders.write.mode("overwrite").save_as_table("DE_SAMPLE_DB.ANALYTICS.fact_orders")

print("Successfully loaded tables into ANALYTICS schema.")
session.table("DE_SAMPLE_DB.ANALYTICS.fact_orders").show()
```

```
Successfully loaded tables into ANALYTICS schema.
-------------------------------------------------------------------
|"ORDER_ID"  |"CUSTOMER_NAME"  |"ORDER_DATE"  |"AMOUNT"  |"STATUS"   |
-------------------------------------------------------------------
|1           |ALICE SMITH      |2026-03-01    |150.5     |COMPLETED  |
|2           |BOB JONES        |2026-03-01    |99.99     |PENDING    |
|3           |CHARLIE BROWN    |2026-03-02    |250.0     |COMPLETED  |
|5           |DIANA PRINCE     |2026-03-03    |300.0     |COMPLETED  |
-------------------------------------------------------------------
```

Step 5: Data Quality & Validation Checks

Verify row counts and aggregate metrics to ensure pipeline integrity.

[12]

```
fact_table = session.table("DE_SAMPLE_DB.ANALYTICS.fact_orders")

total_orders = fact_table.count()
total_revenue = fact_table.agg(spark_sum(col("amount"))).collect()[0][0]
```

[13]

```
print(f"--- Pipeline Validation Report ---")
print(f"Total Valid Orders Loaded: {total_orders}")
print(f"Total Revenue Calculated: ${total_revenue:.2f}")
```

```
--- Pipeline Validation Report ---
Total Valid Orders Loaded: 4
Total Revenue Calculated: $800.49
```

Next Steps

To expand this project further:

- Connect an external stage (Amazon S3 or Azure Blob) using a storage integration to ingest daily CSV/Parquet files automatically.
- Orchestrate this notebook using Snowflake Tasks or integrate it into an Apache Airflow pipeline using the SnowflakeOperator.